Where Hidden Rhythms in Silent Vectors Lie: A Quest into the Emergence of Structures in Character-Level GPT Models

Guillaume Wisniewski
guillaume.wisniewski@u-paris.fr

March 2026

A friendly warning This lab is more ambitious than the previous ones (some colleagues even dared to call it "too ambitious", though I suggested that they might simply be underestimating you), but I hope it will also prove much more interesting. That being said, I am perfectly aware that it cannot realistically be completed in two hours. I therefore strongly encourage you to organise your work so that you do not find yourselves rushing at the last minute, and so that you have enough time to run the training(s) and analyses properly. My hope is that you will learn as much from this lab as I imagine you might and, above all, that you will enjoy it at least as much as I did while writing the assignment. I am also aware that the description of the lab may occasionally lack clarity (and I would understand if you felt that this statement is something of an understatement). When this happens, do not hesitate to choose the interpretation that seems most reasonable (provided that you justify it) or the one that you find the most interesting. You are of course welcome to send me an email. I may not always reply instantly, but I will do my best to clarify things when needed. Likewise, feel free to let your imagination wander and to propose analyses or directions that I may not have considered. Using one's neurons critically remains one of the best ways to learn, to make progress, and, ideally, to have some fun along the way.

Qui a cuer, si face proece.

Prelude

In recent years, (not so) large language models have demonstrated remarkable abilities in modelling natural language, a fact I hope you are now fully convinced of. Yet much remains mysterious about how linguistic structure actually emerges during training (another point on which I also hope I have been more than sufficiently clear).

In this "small" project, you will embark on a somewhat "scientific" quest. Your task will be, nothing less than, to construct and train a character-level GPT model (i.e. a decoder-only Transformer) capable of generating poetry, because we can surely all agree that this world could use a little more poetry, and because I secretly dream of the day when "I" might be able to write my lab assignments in proper alexandrines.

Beyond this already rather ambitious goal, you will also investigate how structure gradually appears in the model's internal representations and how different architectural choices influence this process.

Rather than treating the model as a black box (because you are no longer children and, at the risk of repeating myself, I have no intention of turning you into coding monkeys), you will implement its core mechanisms yourselves, train it on a modest (or perhaps not so modest) corpus of poems, and analyse how linguistic properties emerge inside its internal representations.

Your Quest

Your mission consists of four stages:

1. Implement a Transformer decoder architecture similar to GPT.
2. Train the model on a poetry corpus; vous pouvez remarquer que je n'ai même pas défini d'étape pour la création du corpus tellement je suis convaincu que vous maitrisez maintenant ce type de processus.
3. Monitor the evolution of the model during training.
4. Investigate the emergence of linguistic structure using probing experiments.

The ultimate goal is not merely to obtain a working generator, but to "understand" *how* linguistic structure arises during learning.

Corpus As said before, you will train the model on a corpus of poetry. You are free to choose whichever poet, poetic style, or even language you prefer, provided, of course, that you are willing to wrestle with the joys of Unicode. It is such a fascinating problem that I briefly considered asking you to train a multilingual model using several alphabets, but we sadly have to save that adventure for next year.

Do try, however, to select poetry written in verse, and preferably organised into stanzas (my own knowledge is unfortunately too limited to know whether this is truly universal). This structure will make it easier to observe the organisation of the poem (rhyme, stanza structure, line length, and so on) and to study whether such patterns emerge (or fail to emerge) in the model.

The main practical issue you should pay attention to is the size of the corpus you collect. A simple guideline is the following:

Corpus size	**Purpose**
1–2 MB	quick experimentation
5–10 MB	main experiments
10–20 MB	optional extended training

When preparing the corpus, make sure to preserve:

- line breaks,
- stanza boundaries,
- punctuation.

These structures are essential for studying poetic form.

Stage I: Building the Model Since one has to begin somewhere, and the creation of the corpus can hardly be considered a proper "step", the first step of your quest will consist in implementing a character-level autoregressive Transformer (i.e. a decoder-only model or a GPT-model). The main motivation for working at the character level is to keep the vocabulary extremely small (thereby sparing you the many joys of tokenisation) and to obtain models of a reasonable size, so that you can train them without access to a large GPU cluster.

Like any self-respecting model, your implementation should contain the usual components of a Transformer decoder: token embeddings, positional embeddings, multi-head causal self-attention, feed-forward blocks, residual connections, and layer normalisation. This should go without saying, but I will repeat it anyway: when I say "implement", I of course mean "reuse existing building blocks". There is little value in reimplementing the matrix multiplications underlying the attention mechanism.

However, I do ask you to pay particular attention to the implementation of attention and to choose an efficient approach. This is partly because we have spent quite some time discussing these techniques, and I would like to maintain a certain pedagogical continuity. The recommended implementation relies on `torch.nn.functional.scaled_dot_product_attention`. This function automatically uses `FlashAttention` when it is available. By now you should be sufficiently familiar with the joys of installing Python libraries for this not to

be too much of an obstacle. The idea is to compare this implementation with a more traditional approach based on masked attention with a triangular causal mask, and to measure whether the two implementations differ in terms of training speed and memory consumption.

Another point that deserves careful attention is the choice of a strategy for positional embeddings. Three options are commonly considered: learned positional embeddings, sinusoidal positional encodings, and, for the more adventurous among you, rotary positional embeddings (RoPE). Although I am sure many of you would enjoy implementing all three and comparing their performance, it would probably be more reasonable to choose one (and justify your choice) and keep some energy for the other stages of your noble quest.

Stage II: Training the Model The second step is "simply" to train the model you have just implemented on the corpus you collected. I am sorely tempted to stop the description here and take quiet satisfaction in having optimised the ratio between the time you will spend on this task and the number of characters in the instructions. However, in order to save you from losing too much time, here are a few guidelines (which you are, of course, perfectly free to ignore).

In particular, the following configuration should provide a reasonable starting point, allowing you to obtain a model that is not entirely terrible without exhausting your Google Colab compute budget.

Parameter	**Suggested value**
context length	256
number of layers	6
hidden dimension	256
attention heads	8
MLP expansion	4x
batch size	32–64
learning rate	3×10^{-4}
optimizer	AdamW
dropout	0.1
training steps	20k–50k

Training times will of course depend on your exact configuration, the size of the corpus, and the generosity (or lack thereof) of the Colab GPU assigned to you. As a very rough indication, a model of the size suggested above should typically require between roughly 30 minutes and two hours to complete 20k–50k training steps. This estimate is sufficiently imprecise that you should not rely on it too heavily, but it should at least give you a sense of the order of magnitude and help you organise your experiments.

Because training takes time, it is essential to monitor what is happening during the learning process. At the very least, you should track the evolution of the training and validation losses, and ideally derive a metric such as bits-per-character. In addition, you should periodically generate short samples of poetry from the model. Observing how these generations evolve during training is often surprisingly informative: models tend to move from near-random character sequences to something that gradually begins to resemble words, lines, and sometimes even vaguely poetic structures.

A practical strategy is to log training statistics every few hundred steps (for example every 100–500 steps), evaluate on the validation set at similar intervals, and generate samples every 1k–2k steps. Saving occasional checkpoints is also strongly recommended, both for analysis and in case something goes wrong.

You may also find it useful to employ a short learning-rate warm-up (for instance over the first few hundred or thousand steps) before reaching the nominal learning rate. This is not strictly necessary but often helps stabilise the early stages of training.

The suggested batch size should normally fit on a Colab GPU, but if you encounter memory issues do not hesitate to reduce it. A batch size of 32 is usually a safe fallback, and gradient accumulation can also be used if needed.

Finally, the proposed context length (256) is not arbitrary. You should reflect on whether this value is appropriate for modelling poetic structure. In particular, consider whether it is long enough to capture relationships between lines, rhymes, or stanza patterns in your corpus. You are encouraged to justify this choice explicitly in your report.

Stage III: Exploring the Space of Hyperparameters From the initial configuration proposed above, you should explore the impact of a small number of hyperparameters. In practice, this means selecting at least two or three parameters and studying how changing them affects the behaviour of the model. Obvious candidates include the number of layers, the model dimension, the number of attention heads, the context length, the learning rate, or the amount of dropout.

I obviously cannot prevent you from exploring the entire hyperparameter space if you feel so inclined. However, I would gently draw your attention to the fact that your days still contain only 24 hours, that you probably have other courses to work on (even though I am fully aware that this one — and in particular this lab, and indeed this very question — is by far the most interesting; this is humour, please do not start a war with my colleagues, whom I like very much), and that you may have many other things to explore in life.

More seriously, what interests me here is not that you run a chaotic collection of experiments simply to produce as many numbers as possible without offering any interpretation — that is not science. What I would much rather see is a convincing attempt to understand what actually happens when you vary a given parameter, and to explain

why the behaviour of the model changes in the way that it does.

The goal of this exploration is therefore not merely to report which configuration achieves the lowest loss. You should also reflect on the trade-offs involved: how the different choices affect the final performance, how stable the training process appears to be, and what computational cost they imply. In other words, try to develop a small experimental narrative explaining what you observe and why it might be happening.

Stage IV: Analysing the Poems Generated by the Model *Keep going — we are almost at the final boss.*

Once the model has been trained, the final stage of the quest consists in examining the poems it produces. The goal here is not simply to admire the model's creativity (or lack thereof), but to analyse whether the statistical and structural properties of the training corpus reappear in the generated texts.

Poem generation follows the standard autoregressive procedure used by language models. Starting from an initial prompt, the model predicts a probability distribution over the next possible characters. One then samples a character from this distribution and appends it to the current sequence. The updated sequence is fed back into the model, which predicts the distribution for the next character, and the process is repeated until a desired length is reached.

In practice, it is often helpful to initialise the generation with a short prompt taken from the training corpus. For instance, you may provide the model with a few lines (or verses) of an existing poem and ask it to continue the text. This typically helps the model maintain a coherent structure and encourages the continuation to follow the stylistic and rhythmic patterns already present in the prompt. You may of course also experiment with shorter prompts, or even with an empty prompt, although the resulting poetry may occasionally resemble the avant-garde rather more than intended.

It is often useful to control the randomness of the generation process. This can be done, for instance, by adjusting the *temperature* of the probability distribution (which controls how deterministic or creative the sampling process is), or by restricting sampling to the most likely characters using strategies such as top-k or nucleus sampling. Exploring different settings may produce noticeably different types of text.

Once you have generated a sufficiently large collection of poems, the next step is to analyse their structure. In particular, you should attempt to determine whether the model reproduces some of the statistical regularities present in the training corpus. Depending on the poetic tradition you are working with, this may include examining the typical number of syllables per verse, the length and regularity of stanzas, the distribution of line lengths, or the presence of rhyme patterns at the ends of lines. The precise analyses will naturally depend on the corpus you selected earlier, and you are encouraged to choose measures that make sense for that particular corpus.

The aim of this final stage is therefore not merely to present a few amusing examples

of generated poems, but to investigate, in a systematic way, whether the structural characteristics of the training corpus emerge again in the texts produced by the model. As in the previous stages, what matters most is not the quantity of plots or numbers you produce, but the clarity of the reasoning that connects them.

Work to be submitted

You should submit a short report (between 7 and 10 pages at most) in PDF format, together with a link to a GitHub repository containing the code you developed for this project.

The report should first describe the model you implemented and the main design choices you were led to make along the way. It should also discuss the difficulties you encountered during the implementation or training process, and how you eventually managed to overcome them (ideally with the same courage that has carried you through the previous stages of this quest).

You should then present the training process itself, including the learning curves, and explain how you determined whether the training was proceeding successfully or not. Particular attention should be given to the hyperparameters you explored: describe which ones you varied, what effects you observed, and whether these effects matched your initial intuition.

Finally, you should analyse the poems generated by your model. In particular, try to illustrate whether some of the structural properties of the training corpus appear again in the generated texts. The report should include one or two generated poems, accompanied by a brief commentary discussing what they reveal about the behaviour of the model.

Clarity of reasoning will be valued far more than the sheer quantity of experiments or figures you produce.

Epilogue

If your quest succeeds, you will not only have built a language model from scratch, but also glimpsed how structure gradually arises within neural representations.

Somewhere between random characters and structured verse, hidden rhythms begin to form — and that is magical

Your task is to find them.